

LAB-3-LFT- INSTRUMENTUL LEX

1. Scopul lucrării	
	• studiul generatorului de analizare lexicală LEX • efectuarea unor exercitii practice de utilizare a acestuia
2. Notiuni teoretice	
2.1. Introducere	
	LEX-ul - o este un generator de programe pentru procesarea lexicală a unor fluxuri de intrare. - o acceptă o specificație de nivel înalt, orientată pe problema potrivirii sirurilor de caractere și produce un program într-un limbaj de programare universal. - o Sursa de intrare pentru LEX e un fișier ce conține expresii regulate și fragmente de program asociate acestora. - o La iesire rezultă o funcție numită yylex(), care poate citi un flux de intrare partitionind-ul în siruri de caractere ce se potrivesc cu expresiile regulate date. - o Pe măsura ce fiecare expresie apare în intrarea programului scris de LEX, fragmentul de cod corespunzător este executat. Funcția yylex() este implementarea unui automat finit determinist. - o LEX-ul poate fi folosit singur pentru transformări simple, sau împreună cu alte instrumente pentru analiză și statistică la nivel lexical. - o El poate fi combinat cu un generator de "parser"-e (analizare sintactice), cum e de exemplu YACC-ul. În acest caz yylex() e un preprocesor lexical ce recunoaște simbolurile terminale ("token"-ii). YACC-ul • generează o funcție, yparse(), ce atribuie pieselor lexicale (rezultate din yylex()) o structură conform unei gramatici independente de context. • Fluentele controlului într-un asemenea caz este: Intrare --> yylex() --> yyparse() --> Intrare analizată sintactic. • Automatul generat este interpretat, nu compilat, în ideea de a salva spațiu. • Rezultatul e totuși un analizor rapid. • Mai concret, timpul cerut de un program generat de LEX pentru a recunoaște și a partitiona un flux de intrare, e proporțional cu lungimea intrării. • Numărul de reguli LEX, sau complexitatea lor, nu sunt importante în determinarea vitezei, dacă regulile ce includ context înainte nu cer o cantitate semnificativă de re-scanare. • Ceea ce crește cu numărul și complexitatea regulilor, este dimensiunea automatului, deci dimensiunea programului generat de LEX. • Fragmentele de cod utilizator sunt aranjate ca și "case"-uri într-un "switch". • Utilizatorul e liber să insereze declarații și instrucțiuni adiționale în rutina ce conține acțiunile, sau să adauge subrutine în afara acestor rutine-acțiune. • LEX-ul nu e limitat la surse ce pot fi interpretate pe baza unui singur caracter "look-ahead" (privește-inainte). • De exemplu, dacă există două reguli, una cautând ab iar alta cautând abcdefg , și fluxul de intrare este abcdefh , LEX-ul va recunoaște ab și va lăsa pointerul înaintea lui cd... O asemenea revenire e mai costisitoare decât este necesar în procesarea limbajelor mai simple.

LAB-3-LFT- INSTRUMENTUL LEX

2.2. Sursa LEX	Formatul general al sursei LEX este : <pre> { definitii } %% { reguli } %% { subrutine utilizator } </pre> - Definitiiile si subrutinele utilizator pot lipsi. - Al doilea %% e optional, dar primul e obligatoriu pentru a marca inceputul regulilor. - Cea mai scurta sursa pentru LEX arata astfel: %% fara nici o definitie, nici o regula. o Aceasta sursa e translatata intr-un program care copiaza intrarea la iesire. o Regulile reprezinta deciziile de control ale utilizatorului. Ele formeaza o tabela in care coloana stinga contine expresii regulate, iar coloana dreapta contine actiuni, fragmente de program ce trebuie executate cind expresiile sint recunoscute. o O regula pentru a cauta sirul <code>integer</code> in fluxul de intrare, si a tipari "found keyword INT" oricind acesta e gasit, poate fi: <pre> integer printf("found keyword INT"); </pre> o In acest exemplu, limbajul gazda este C-ul. o Sfirsitul expresiei este indicat de primul caracter "blanc" sau "tab".
2.3. Expresii regulate LEX	
2.3.1. Caractere text si caractere operator.	O expresie regulata - specifica un set de siruri de cautat/potrivit. - Ea contine caractere text, ce corespund caracterelor din sirurile de cautat, si caractere operator, ce specifica repetitii, alegeri si alte proprietati. - Literele alfabetului si cifrele sint intotdeauna caractere text. Operatorii sint: <pre> " \ [] ^ - ? . * + () \$ / { } % < > </pre> o Cind se doreste utilizarea lor ca si caractere text, operatorii trebuie "protejati". o Operatorul <code>"</code> arata ca orice caractere continute intre o pereche de ghilimele, vor fi tratate ca si caractere text. o Un caracter operator mai poate fi protejat precedindu-l de un caracter <code>\</code> ca in: <code>xyz\+ \+</code> care este echivalent cu <code>xyz"++"</code>, si cu <code>"xyz++"</code>. In mod normal, "blanc"-urile si "tab"-urile incheie o regula. Sint recunoscute: <code>\n (newline)</code>, <code>\t (tab)</code>, <code>\b (backspace)</code>. o Pentru a introduce caracterul <code>\</code> se foloseste <code>\\</code>.
2.3.2. Clase de caractere	o Perechea de operatori <code>[]</code> defineste clase de caractere. o De exemplu <code>[abc]</code> se potriveste cu un caracter, care poate fi a, b sau c. o Intre paranteze drepte semnificatiile celor mai multi operatori sint ignorate. o Exista insa operatorii speciali: <code>\</code> - si <code>^</code>. Caracterul <code>-</code> indica domenii. o De exemplu <code>[a-z0-9<>]</code> se potriveste cu orice caracter din multimea continind literele mici, cifrele si parantezele unghiulare. Domeniile pot fi date in orice ordine. o Daca se doreste includerea caracterului <code>-</code> intr-o clasa de caractere, el trebuie sa fie primul sau ultimul <code>[-+0-9]</code> se potriveste cu toate cifrele si cele doua semne <code>+</code> <code>-</code>. o Operatorul <code>^</code> trebuie sa apara primul dupa <code>[</code>. El indica faptul ca sirul rezultat trebuie sa fie complementat fata de setul de caractere al calculatorului. o Astfel: <code>[^abc]</code> se potriveste cu toate caracterele cu exceptia lui a, b si c. <code>[^a-zA-Z]</code> reprezinta orice caracter care nu e o litera. o Caracterul <code>\</code> furnizeaza "escape"-urile uzuale intre clasele de caractere dintre paranteze drepte. Secventele escape in octal sint legale, dar ne-portabile: <code>[\40-\176]</code> se potriveste cu toate caracterele din setul ASCII, de la octal 40 (blanc) pina la octal 176 (tilda).
2.3.3. Caracter arbitrar.	Caracterul operator <code>.</code> se potriveste cu orice caracter exceptind "newline".

LAB-3-LFT- INSTRUMENTUL LEX

2.3.4. Expresii optionale	Operatorul <code>?</code> indica optionalitatea unui element. Astfel: <code>ab?c</code> se potriveste cu <code>ac</code> sau cu <code>abc</code> ('b' fiind optional).
2.3.5. Expresii repetate	- o Repetițiile sint indicate de operatorii <code>*</code> si <code>+</code>. - o De exemplu: <code>a*</code> reprezinta zero sau oricite aparitii ale lui <code>a</code>, <code>a+</code> reprezinta una sau oricite aparitii ale lui <code>a</code>, <code>[a-z]+</code> reprezinta orice sir de litere mici (mai puțin sirul vid) si <code>[A-Za-z][A-Za-z0-9]*</code> reprezinta toate sirurile alfanumerice cu un caracter alfabetic in fata.
2.3.6. Alternare si grupare	- o Operatorul <code> </code> indica alternare: <code>(ab cd)</code> se potriveste fie cu <code>ab</code>, fie cu <code>cd</code>. - o De notat ca parantezele sint folosite pentru grupare, cu toate ca ele nu sint necesare la nivelul exterior unde <code>ab cd</code> este suficient. - o Parantezele pot fi folosite pentru expresii mai complicate, ca de exemplu <code>(ab cd+)?(ef)*</code>, care se potriveste cu siruri ca <code>abefef</code>, <code>efefef</code>, <code>cdef</code>, <code>cddd</code>, dar nu si cu <code>abc</code>, <code>abcd</code> sau <code>abcdef</code>.
2.3.7. Dependente de context.	- o LEX-ul recunoaste si o anumita cantitate de context inconjurator. - o Cei mai simpli operatori pentru acest lucru sint <code>^</code> si <code>\$</code>. - o Daca primul caracter dintr-o expresie este <code>^</code>, expresia va fi recunoscuta numai la inceputul unei linii. - o Daca ultimul caracter este <code>\$</code> expresia va fi potrivita numai cind este urmata de un "newline". - o Un operator special, <code>/</code>, indica context "dira" (lasat in urma sau continuare). - o Astfel, <code>ab/cd</code> se potriveste cu sirul <code>ab</code> numai daca e urmat de <code>cd</code>, iar <code>ab\$</code> e identic cu <code>ab/n</code>. - o Contextul stnga e tratat in LEX prin conditii de start. - o Daca o regula trebuie sa fie executata numai cind automatul LEX e in conditia de start <code>x</code>, regula trebuie prefixata de <code><x></code>, folosind operatorul <code><></code>. - o Daca luam "a fi la inceputul liniei" ca fiind conditia de start ONE, atunci operatorul <code>^</code> ar fi echivalent cu <code><ONE></code>.
2.3.8. Repetitii si definitii	- o Operatorii <code>{ }</code> specifica repetitii sau expandari de definitii. - o De exemplu: <code>{digit}</code>, cauta un sir predefinit numit <code>digit</code> si il expandeaza in punctul respectiv, iar <code>a{1,5}</code> cauta una pina la cinci aparitii ale lui <code>a</code>. Un <code>%</code> initial separa segmentele sursa LEX.

2.4. Actiuni LEX

	- o La identificarea unei expresii regulate in intrare, programul generat de LEX executa actiunea corespunzatoare. - o Exista o actiune implicita: copierea intrarii la iesire, actiune ce e efectuata si asupra tuturor sirurilor ce nu sint potrivite cu nici o expresie. - o Astfel, daca se doreste sa nu scape nimic "nefiltrat" in iesire, trebuie sa se furnizeze reguli care sa recunoasca totul. - o Aceasta e situatia uzuala in cazul in care LEX-ul e folosit impreuna cu YACC-ul. - o Deoarece se poate considera ca actiunile sint ceea ce se face in loc de a copia intrarea la iesire, in general, o regula care doar copiaza, poate fi omisa. - o regula des intilnita e: <code>[\t\n]</code>; ea eliminind spatiile, "tab"-urile si "newline"-urile. - o Caracterul <code> </code> indica faptul ca actiunea pentru regula respectiva e definita la urmatoarea regula. - o Exemplul precedent poate fi scris si astfel: <pre> " " "\t" "\n" ; rezultatul fiind acelasi. </pre> - o In actiuni mai complexe, se doreste adesea sa se stie sirul curent potrivit cu o expresie. - o LEX-ul lasa acest text intr-un tablou extern de caractere numit <code>yytext</code>. - o Astfel, regula: <code>[a-z]+ printf("%s",yytext);</code> va tipari sirul din <code>yytext</code>. - o Aceasta actiune poate fi scrisa ca ECHO: <code>[a-z]+ ECHO;</code>
--	--

LAB-3-LFT- INSTRUMENTUL LEX

- o Uneori e convenabil sa se cunoasca sfirsitul sirului gasit. LEX-ul furnizeaza un contor, `yyleng` , al numarului de caractere potrivite.
- o Pentru a contoriza atit numarul de cuvinte, cit si numarul de caractere din cuvintele de la intrare, se poate scrie regula: `[a-zA-Z]+ {words++; chars+=yyleng;}` ,regula ce aduna in `chars` numarul de caractere din cuvintele recunoscute, numarate in `words`.
- o Ultimul caracter din sirul potrivit poate fi accesat prin: `yytext[yyleng-1]`.
- o Ocazional, o actiune LEX poate decide ca o regula nu a recunoscut sirul de caractere corect.
- o Pentru aceasta situatie sint furnizate doua rutine:

- **yymore()** ,ce poate fi apelata pentru a indica faptul ca urmatoarea expresie de intrare recunoscuta trebuie sa fie adaugata la sfirsitul ultimului sir de caractere recunoscut (in mod normal, urmatorul sir de intrare suprascrie sirul curent din `yytext`), si
- **yyless(n)** care indica faptul ca nu toate caracterele potrivite de expresia curenta sint dorite acum. Argumentul `n` indica numarul de caractere de retinut din `yytext`, restul fiind returnate la intrare. Acest lucru ofera acelasi gen de "privire inainte" ca cel oferit de operatorul `/` , dar intr-o forma diferita.
- De exemplu, sa consideram un limbaj ce defineste un sir ca fiind un set de caractere intre ghilimele, si cere ca aparitia unui caracter " (ghilimele) intr-un sir sa fie precedata de `\` . Expresia regulata ce descrie acest lucru e oarecum confuza, astfel incit e preferabil sa se scrie regula:

```
\"[^\"]* { if(yytext[yyleng-1]=="\\") yymore();
           else <<...procesarea dorita de utilizator>>
         }
```

care atunci cind va intilni un sir ca `"abc\"def"` , va potrivi primele 5 caractere `"abc\"` , dupa care apelul la `yymore()` va face sa fie adaugata cealalta parte a sirului, adica `"def` .

- o De notat ca semnul ghilimele final, care termina sirul trebuie sa fie tratat ("ciugulit") in codul `<<...procesarea dorita de utilizator>>`.

o Functia **yyless()** poate fi folosita pentru a reprocesa text in diferite circumstante.

- o Considerind problema ce apare in limbajul C de a distinge ambiguitatea lui `"=-a"` , sa presupunem ca aparitia se doreste a fi tratata ca `"=- a"` , si sa se tipareasca si un mesaj. O rezolvare poate fi :

```
=-[a-zA-Z] { printf("Operator (=-) ambiguu\n");
             yyless(yyleng-1);
             ... actiune pentru -= ...
           }
```

solutie ce tipareste mesajul, returneaza litera de dupa operator in fluxul de intrare, si trateaza operatorul drept `"=-"`. Alternativ, poate fi de dorit sa se trateze aparitia `"=-a"` ca `"=- a"`. Pentru a realiza acest lucru se returneaza semnul minus si litera de la intrare :

```
=-[a-zA-Z] { printf("Operator (=-) ambiguu\n");
             yyless(yyleng-2);
             ... actiune pentru = ...
           }
```

- o De notat ca expresiile pentru cele doua cazuri pot fi scrise mai usor astfel: `=-/[A-Za-z]` in primul caz, si `=-/[A-Za-z]` ,in al doilea caz, rezolvare in care nu trebuie efectuata nici o revenire in actiunea regulii.

o Nu e necesar sa fie recunoscut intregul identificator pentru a observa ambiguitatea.

- o Oricum, posibilitatea `"=-3"` , face ca: `=-/[\t\n]` ,sa fie o regula mai buna.

LEX-ul permite accesul la **rutinele de I/O** ce le foloseste. Ele sint:

- 1) **input()** -returneaza urmatorul caracter din intrare
- 2) **output(c)** -scrie caracterul `c` la iesire
- 3) **unput(c)** -pune caracterul `c` inapoi in fluxul de intrare spre a fi citit mai tirziu de `input()`

LAB-3-LFT- INSTRUMENTUL LEX

	- o Implicit, aceste rutine sint furnizate ca macrodefinitii, dar utilizatorul le poate inlocui cu versiuni proprii. Aceste rutine definesc relatia dintre fisierele externe si caracterele interne, si trebuie sa fie pastrate toate sau modificate toate, pentru consistenta. - o Ele pot fi redefinite pentru a face ca intrarea/iesirea sa fie transmise dinspre/inspre alte programe, memoria interna, etc. Setul de caractere folosit trebuie sa fie consistent in toate rutinele. - o Un zero citit la intrare trebuie sa insemne "End_of_file", iar relatia dintre unput() si input() trebuie pastrata, altfel "cautarea inainte" a LEX-ului nu va functiona. - o LEX-ul nu "priveste inainte" deloc daca nu trebuie sa o faca, dar orice regula terminata in +,*,? sau \$, sau continind / , implica acest lucru. - o "Privirea inainte" e necesara de asemenea pentru a potrivi o expresie care e prefixul altei expresii. - o Biblioteca LEX standard impune 100 de caractere drept limita a cantitatii pentru revenire. - o O alta rutina LEX pe care utilizatorul o poate redefini e yywrap() , apelata cind se atinge un "end_of_file". Daca yywrap() returneaza 1, LEX-ul continua cu "wraparea" (terminarea) normala la sfirsitul intrarii. Uneori, e convenabil sa se preia intrare in continuare, de la o noua sursa. In acest caz, utilizatorul trebuie sa furnizeze un yywrap() care furnizeaza noua intrare, returneaza 0 si informeaza LEX-ul sa continue procesarea. Implicit yywrap() returneaza 1. Aceasta rutina e de asemenea un loc convenabil de a tipari tabele, sumare, etc., la sfirsitul programului.
--	--

2.5. Reguli ambigue

	LEX-ul poate manipula si specificatii ambigue. Cind mai mult de o expresie se poate potrivi cu intrarea curenta, LEX-ul decide dupa urmatoarele criterii: 1) alege cea mai lunga potrivire, 2) intre regulile care se potrivesc, si au acelasi numar de caractere, alege regula ce apare prima in tabela de reguli. - o Sa presupunem ca avem regulile: integer actiune pentru cuvint cheie...; [a-z]+ actiune pentru identificator...; date in aceasta ordine. Daca intrarea e integers, ea e tratata ca un identificator, deoarece [a-z]+ se potriveste cu 8 caractere iar integer doar cu 7. Daca intrarea e integer, ambele reguli se potrivesc cu 7 caractere, si intrarea e tratata drept cuvint cheie, conform regulii ce apare prima. - o Principiul preferintei pentru cea mai lunga potrivire, face regulile ce contin operatorii . si * relativ periculoase. De exemplu: '.*' , pare o buna maniera de a specifica un sir intre apostroafe, insa practic e o invitatie pentru program sa citeasca mult inainte, cautind un apostrof indepartat. - Daca la intrare avem: 'first' quoted string here, 'second' here, expresia de mai sus se va potrivi cu 'first' quoted string here, 'second' , ceea ce probabil ca nu e ceea ce s-a dorit. - O regula mai buna e: '[^\\n]*' care pentru intrarea de mai sus se va opri dupa 'first'. - o Consecintele unor erori ca acestea sint restrinse de faptul ca operatorul . (punct) nu se va potrivi cu "newline". - o Astfel, expresii ca .* se opresc pe linia curenta. Nu incercati sa "invingeti" aceasta cu expresii ca [.\\n]+ sau echivalente, deoarece programul generat de LEX va incerca sa citeasca intregul fisier de intrare si vor rezulta depasiri de buffer intern. - o De notat ca, in mod normal, LEX-ul partitioneaza fluxul de intrare, fara sa caute toate potrivirile posibile ale fiecarei expresii. - o Acest lucru inseamna ca fiecare caracter e contorizat o singura data. - o Sa presupunem de exemplu ca se doreste contorizarea atit a aparitiilor lui he cit si a aparitiilor lui she din intrare. O posibilitate de a realiza acest lucru este:
--	---

LAB-3-LFT- INSTRUMENTUL LEX

	<pre> she s++; he h++; \n . ; </pre> unde ultimele doua reguli ignora orice altceva in afara de he si she. - o Sa ne amintim ca . nu include "newline". Pentru ca she include he , in mod normal LEX-ul nu va recunoaste instantierile lui he incluse in she , deoarece o data ce a trecut de o instantiere she , respectivele caractere sint pierdute. - o Uneori, utilizatorul doreste sa modifice acest comportament implicit. Actiunea REJECT inseamna "du-te si executa urmatoarea alternativa". In acest caz e executata regula care e urmatoarea alternativa posibila (dupa regula curenta). - o Pozitia pointerului de intrare e ajustata corespunzator. Sa presupunem ca utilizatorul doreste sa contorizeze si instantierile lui he incluse in she. Urmatoarele reguli reprezinta un mod de a realiza acest lucru: <pre> she {s++;REJECT;} he {h++;REJECT;} \n . ; </pre> In acest exemplu, desigur, utilizatorul poate observa ca she include he , dar nu si viceversa, si poate omite actiunea REJECT pentru he . Sint cazuri insa, in care s-ar putea sa nu se stie "a priori" ce caractere sint in ambele clase.
--	--

2.6. Definitii in sursa LEX

	Uneori utilizatorul are nevoie de optiuni aditionale, cum ar fi definirea unor variabile in scopul utilizarii lor de catre programul propriu sau de catre LEX. Acestea pot sa apara fie in sectiunea de definitii fie in sectiunea de reguli. Sa ne amintim ca LEX-ul transforma regulile intr-un program. Orice cod sursa, neinterceptat de LEX, e copiat in programul generat. Exista 3 clase de asemenea cazuri: 1) Orice linie ce nu e parte a unei reguli sau actiuni LEX si care incepe cu "blanc" sau "tab", e copiată in functia generata de LEX. Astfel de cod sursa, daca apare inaintea primului delimitator %% , va fi extern oricarei functii din programul generat. Cind codul urmeaza imediat primului %% , el va fi pus in partea de declaratii a functiei generate de LEX . Acest cod sursa trebuie sa preceada prima regula LEX. Ca un efect lateral, liniile ce incep cu "blanc" sau "tab", si care contin un comentariu, sint trecute programului generat. Acest lucru poate fi folosit pentru a include comentarii atat in sursa LEX, cit si in codul generat. Comentariile trebuie sa urmeze conventiile limbajului gazda. 2) Codul sursa inclus intre liniile ce contin doar %{ si %} e copiat in programul generat. Delimitatorii sint ignorati. Acest format permite introducerea de text, cum ar fi de exemplu instructiunile preprocesor ce trebuie sa inceapa in coloana 1, sau linii ce nu par fragmente de programe. 3) Codul sursa ce apare dupa al doilea delimitator %% este copiat la sfarsitul codului generat de LEX. - o Definitiiile pentru LEX trebuie date inainte de primul delimitator %% . Orice linie din aceasta sectiune, necontinuta intre %{ si %} , si incepind in coloana 1, e presupusa a defini siruri de substitutie LEX. Formatul unei asemenea linii este :name translation iar efectul ei e ca sirul dat ca translatare, translation e asociat cu numele name. Numele si translatarea trebuie sa fie separate prin cel putin un spatiu sau un "tab", iar numele trebuie sa inceapa cu o litera. Traslatarea poate fi apelata (expandata) utilizind {name} in locul unde se doreste sa apara translation. Sectiunea de definitii poate contine si alte comenzi, inclusiv selectia limbajului gazda, o tabela cu setul de caractere, o lista de conditii de start iar pentru programe sursa mari expandari ale tablourilor LEX-ului fata de dimensiunea implicita.
--	---

2.7. Utilizare

Exista doua etape in realizarea unui program executabil, pornind de la o sursa LEX:

- 1) **translatarea sursei de catre LEX** intr-un program generat in limbajul gazda, (cu comanda:
lex [-acntv] [-o nume_fis.c] nume_fis_sursa.l);
- 2) **compilarea programului generat**, de obicei impreuna cu o biblioteca de subrutine LEX.
(bcc nume_fis.c \lib\tslex.lib)

2.8. Dependente de context stinga

- o Uneori, e de dorit sa avem mai multe seturi de reguli lexicale aplicabile intrarii la diferite momente. De exemplu, un preprocesor dintr-un compilator, ar putea distinge instructiunile preprocesor, pentru a le analiza altfel decit instructiunile ordinare.
- o Acest caz necesita sensibilitate la context precedent, si poate fi tratat in mai multe moduri.
- o Operatorul $\wedge$, de exemplu, e un operator de context precedent si recunoaste contextul stinga imediat precedent, asa cum $\$$ recunoaste contextul dreapta imediat urmator.
- o Contextul stinga adiacent, poate fi extins pentru a produce o facilitate similara cu cea pentru contextul dreapta adiacent, dar e putin probabil ca acest lucru sa fie util, deoarece deseori contextul stinga relevant a aparut la un moment precedent, cum ar fi de exemplu inceputul liniei.
- o Exista trei **cazuri de tratare a diferitelor medii** (contexte sau "environments"):
 - [1] utilizarea simpla a flag-urilor, cind doar citeva reguli "schimba" dint-un mediu intr-altul,
 - [2] utilizarea conditiilor de start asupra regulilor,
 - [3] posibilitatea ca mai multe analizoare lexicale sa ruleze impreuna.
- o In fiecare caz, exista reguli ce recunosc necesitatea de a schimba mediul in care urmatorul text de intrare sa fie analizat, si necesitatea de a seta un parametru pentru a reflecta schimbarea. Acest parametru poate fi un flag testat explicit de codul din actiunea utilizator. Un asemenea flag reprezinta modalitatea cea mai simpla de a solutiona roblema, deoarece nu e implicat LEX-ul. Uneori poate fi mai convenabil ca LEX-ul sa memoreze flag-urile ca si conditii initiale asupra regulilor. Orice regula poate fi asociata cu o conditie de start, caz in care ea va fi recunoscuta numai cind LEX-ul este in respectiva conditie de start. Conditia de start curenta poate fi modificata in orice moment.
- o Daca seturile de reguli pentru diversele medii sint foarte diferite, pentru a obtine claritate e bine sa se scrie mai multe analizoare lexicale distincte, asigurindu-se comutarea de la unul la altul dupa necesitati.
- o Sa consideram problema copierii intrarii la iesire, cu modificarea cuvintului "magic" in "first" in fiecare linie ce incepe cu litera a, in "second" in fiecare linie ce incepe cu litera b, si in "third" in fiecare linie ce incepe cu litera c. Problema e foarte simpla, deci putem folosi doar un flag :

```

int flag;
%%
^a    { flag='a' ; ECHO; }
^b    { flag='b' ; ECHO; }
^c    { flag='c' ; ECHO; }
\n    { flag= 0 ; ECHO; }
magic { switch(flag) {
        case 'a': printf("first");break;
        case 'b': printf("second");break;
        case 'c': printf("third");break;
        default : ECHO; break;
    }
}
    
```

LAB-3-LFT- INSTRUMENTUL LEX

	o Pentru a rezolva problema folosind conditii de start, fiecare conditie de start trebuie mentionata in sectiunea de definitii printr-o linie de forma: %start name1 name2 Cuvintul start poate fi inlocuit cu s sau S o Conditiiile pot fi referite la inceputul unei reguli folosind parantezele unghiulare (<>). De exemplu: <name1>expression e o regula recunoscuta doar atunci cind LEX-ul e in conditia de start name1. o Activarea unei conditii de start, se face cu instructiunea actiune: BEGIN name1; , care schimba conditia de start in name1. Revenirea la starea normala se face cu instructiunea: BEGIN 0; care reseteaza conditia initiala a automatului LEX. O regula poate fi activa in mai multe conditii de start, deci: <name1,name2,name3> e un prefix legal. Orice regula care nu incepe cu operatorul prefix <> e activa permanent. Problema de mai sus poate fi rezolvata si astfel: <pre> %START AA BB CC %% ^a { ECHO; BEGIN AA; } ^b { ECHO; BEGIN BB; } ^c { ECHO; BEGIN CC; } \n { ECHO; BEGIN 0; } <AA>magic printf("first"); <BB>magic printf("second"); <CC>magic printf("third"); </pre> Logica e aceeaasi ca in solutia precedenta, acum insa e folosit LEX-ul si nu codul utilizatorului.
--	---

2.9. Setul de caractere

	o Programele generate de LEX trateaza intrarea/iesirea la nivel de caracter numai prin rutinele input(), output() si unput(). Reprezentarea caracterelor furnizata in aceste rutine e acceptata de LEX si e folosita pentru a returna valori in yytext. Un caracter e reprezentat ca un intreg mic ("short int"), care, atunci cind e folosita biblioteca standard, are o valoare egala cu valoarea intreaga a "sablonului-bit" reprezentind caracterul in calculatorul gazda. o In mod normal, litera a e reprezentata in aceeaasi forma ca si constanta caracter 'a' . Daca se schimba aceasta interpretare, prin furnizarea unor rutine de intrare/iesire ce translateaza caracterele, LEX-ul trebuie informat despre acest lucru printr-o tabela de traducere. o Aceasta tabela trebuie sa apara in sectiunea de definitii, incadrata intre 2 linii "%T" , si ea contine rinduri de forma : {integer}{ character string } ce indica valoarea asociata unui caracter sau unui set de caractere. Astfel, urmatorul exemplu : <pre> %T 1 Aa 2 Bb 26 Zz 27 \n 28 + 29 - 30 0 31 1 39 9 %T </pre> "mapeaza" literele mari si mici impreuna in intregii 1..26, "newline" in 27, + si - in 28 si 29, iar cifrele in 30..39 . Se observa folosirea notatiei "escape" pentru "newline".
--	---

LAB-3-LFT- INSTRUMENTUL LEX

	- o Daca e furnizata o tabela de translatate, fiecare caracter ce poate sa apara in reguli sau in orice intrare valida, trebuie inclus in tabela. - o Atentie, nici unui caracter nu i se poate atribui valoarea 0 , si nici unui caracter nu i se poate atribui o valoare mai mare decat dimensiunea setului hardware de caractere.
--	--

3. Desfasurarea lucrarii

3.1. Se vor executa practic exemplele prezentate in sectiunea 2 a lucrarii, si se va analiza codul generat de LEX.

3.2. Se vor testa urmatoarele exemple:

3.2.1. Eliminarea spatiilor si "tab"-urilor de la sfirsitul liniilor unui text de intrare si inlocuirea celorlalte spatii si "tab"-uri cu un singur spatiu:

```
%%  
[ \t]+$ ;  
[ \t]+ printf(" ");  
%%
```

3.2.2. Identificarea in sirul de intrare a numerelor intregi si reale in conventia FORTRAN:

```
D [0-9]  
E [DEde][-+]?{D}+  
%%  
{D}+ printf("integer");  
{D}+"."{D}*({E})? |  
{D}*"."{D}+({E})? |  
{D}+{E} printf("real");  
%%
```

3.2.3. Realizarea unei histograme dupa lungimea cuvintelor din textul de intrare, unde un cuvint e definit ca un sir de litere:

```
int lung[99];  
%%  
[a-z]+ lung[yyleng]++;  
.  
\n ;  
%%  
yywrap()  
{  
    int i;  
    printf("Lungime Nr. cuvinte\n");  
    for(i=0; i<99; i++) if(lung[i] > 0) printf("%6d %7d\n", i, lung[i]);  
    return(1);  
}
```

4. Intrebari si dezvoltari

4.1. Sa se propuna si alte exemple sau imbunatatiri la cele prezentate.

4.2. Sa se implementeze un exemplu care sa copieze intrarea la iesire adunand 7 la numerele cuprinse in intervalul [7-12], si inmultind cu 10 numerele negative mai mici decat -100.